

# Home SOC Lab Portfolio

Phases 1 – 8: Vulnerability Assessment through Advanced Packet Analysis

---

Built on an existing home SOC lab (Wazuh SIEM, Suricata IDS, Docker, Ubuntu VM, and a Windows 11 host)  
— every finding investigated, verified with evidence, and remediated where required.

Prepared as a personal cybersecurity portfolio project

# Capstone Executive Summary

Over the course of this project, I took my existing home SOC lab — Wazuh SIEM, Suricata IDS, Nmap, a Docker-based Wazuh deployment, and a single Ubuntu VM — and used it as the foundation for eight phases of hands-on security work, moving from basic vulnerability assessment through to full packet-level analysis. Rather than treating each phase as an isolated exercise, I ran them as a connected investigation, where findings from earlier phases directly shaped later ones.

**Phase 1 — Vulnerability Assessment:** I scanned my own lab environment with Nmap, identified a real, current CVE (CVE-2026-60002) affecting my installed OpenSSH client version, and worked through the full remediation lifecycle: checking for a vendor patch (none was available yet), then applying compensating controls — restricting SSH access by subnet via ufw and switching to key-only authentication. Along the way I made and caught my own mistakes (an incorrect firewall rule deletion, a duplicate config directive silently overriding my first fix) and documented the corrections rather than hiding them.

**Phase 2 — Log Analysis:** I used the real log evidence generated by Phase 1's remediation work to practice reading auth.log and ufw.log, distinguishing legitimate administrative activity from anomalies. This surfaced a genuine unexplained finding — a one-time blocked FTP connection to an unfamiliar host — which I investigated as far as the evidence allowed and documented honestly as inconclusive rather than forcing a false conclusion.

**Phase 3 — SIEM Investigation:** I brought that FTP anomaly into Wazuh, traced it to a Suricata signature for a well-known Metasploitable vulnerability, and confirmed my machine was never actually compromised. In the process, I discovered a significant gap in my own SIEM: auth.log had never been configured as a monitored source, meaning real SSH activity had been invisible to Wazuh the entire time. I fixed the configuration and verified the fix with a live test.

**Phase 4 — Incident Response:** I simulated a brute-force SSH attack against my own hardened system, confirmed the Phase 1 controls held (zero successful logins), but discovered the attempt didn't generate a SIEM alert either — a real, documented detection reliability gap tied to intermittent agent connectivity. I walked the incident through detection, investigation, containment, and recovery, even though the underlying finding was inconclusive but real, which I believe is a more honest and useful capstone artifact than a clean, staged success story.

**Phase 5 — Local Security Audit:** Adapting the original Active Directory phase to my actual Windows 11 machine, I audited local accounts and found a genuinely enabled account with no password required. I disabled it, strengthened password history policy, and reviewed the Windows Security event log, confirming no evidence of external unauthorized access.

**Phase 6 — Threat Hunting:** I proactively searched my system for signs of compromise across processes, network listeners, and recent file activity — without waiting for an alert. I found one process ownership anomaly, verified it with UID lookups rather than assuming the worst, and correctly cleared it as a benign Docker artifact. The hunt concluded clean, which I documented as a legitimate result in itself.

**Phase 7 — Packet Analysis:** I stood up a deliberately insecure Telnet service, captured a live login session with Wireshark, and used Follow TCP Stream to recover my own credentials in plaintext — direct, visual proof of exactly why the SSH hardening from Phase 1 mattered. I disabled the Telnet service immediately afterward.

**What this demonstrates:** Across all eight phases, the consistent thread isn't that everything went perfectly — it's that I could investigate methodically, verify claims with evidence rather than assumption, document inconclusive results honestly, and close the loop by fixing what I found. That, more than any single tool, is what I understand real SOC and security analyst work to actually require.

# Table of Contents

1. Phase 1 — Vulnerability Assessment & Nmap
2. Phase 2 — Log Analysis & Security Monitoring
3. Phase 3 — SIEM / SOC Investigation (Wazuh)
4. Phase 4 — Incident Response
5. Phase 5 — Local Security Audit (Windows 11)
6. Phase 6 — Threat Hunting
7. Phase 7 — Advanced Wireshark Investigation
8. Phase 8 — Final Capstone Summary

# Phase 1 — Vulnerability Assessment & Nmap

## Objective

Perform reconnaissance and a controlled vulnerability assessment against the lab's own Ubuntu VM using Nmap, then carry any real finding through to remediation.

## Environment

Kali-style single Ubuntu VM (VirtualBox, host-only network 192.168.100.0/24), target IP 192.168.100.20 (interface enp0s8), Nmap, ufw, OpenSSH.

## Commands Used

```
ip a
```

→ Identifies network interfaces; confirms the lab-facing IP (192.168.100.20) versus the NAT interface.

```
sudo nmap -sV -O 192.168.100.20
```

→ Service/version detection plus OS fingerprinting against the target host.

```
sudo nmap -sV --script ssh2-enum-algos,ssh-auth-methods 192.168.100.20
```

→ Enumerates SSH key exchange/encryption algorithms and accepted authentication methods.

```
sudo nmap -sV --script vuln 192.168.100.20
```

→ Runs Nmap's vulnerability-detection script category against the detected service versions.

```
apt-cache policy openssh-client
```

→ Compares the installed OpenSSH version against the newest version available in the repos.

```
sudo apt update && sudo apt install --only-upgrade openssh-client openssh-server -y
```

→ Upgrades OpenSSH to the latest available package version.

```
sudo ufw status numbered
```

→ Lists current firewall rules with index numbers for precise deletion.

```
sudo ufw allow from 192.168.100.0/24 to any port 22
```

→ Restricts SSH access to the trusted lab subnet only.

```
sudo ufw delete <number>
```

→ Removes a specific (overly permissive) firewall rule by number.

```
ssh-keygen -t ed25519 -C "javy-lab-key"
```

→ Generates a new Ed25519 SSH key pair for key-based authentication.

```
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys &&
chmod 700 ~/.ssh
```

→ Authorizes the new public key and sets the strict permissions SSH requires.

```
sudo nano /etc/ssh/sshd_config (PasswordAuthentication no)
```

→ Disables password-based SSH login, enforcing key-only authentication.

```
sudo systemctl restart ssh
```

→ Applies the new SSH server configuration.

```
ssh vboxuser@192.168.100.20
```

→ Live test confirming key-only login succeeds and password auth is refused.

## Findings

- Only one open port (22/SSH) on the target — minimal attack surface.
- SSH allowed both publickey and password authentication by default.
- CVE-2026-60002 (CVSS 9.4) — OpenSSH client use-after-free vulnerability — identified via Nmap's vuln script, affecting the installed 10.2p1 build.
- No vendor patch was available in the Ubuntu repos at time of assessment (verified via apt-cache policy).

## Remediation / Outcome

- Applied compensating control: restricted SSH access to the trusted 192.168.100.0/24 subnet via ufw (caught and corrected a rule-deletion mistake along the way, illustrating why bottom-up deletion avoids renumbering errors).
- Generated an SSH key pair, deployed it to authorized\_keys, and disabled password authentication.
- Discovered and fixed a duplicate/overriding PasswordAuthentication directive later in sshd\_config that had silently kept password auth active despite the first edit — root-caused via targeted grep and sed, then re-verified with a live login test.

# Phase 2 — Log Analysis & Security Monitoring

## Objective

Read and interpret raw system logs to distinguish normal administrative activity from suspicious or unexplained events, using the real evidence generated by Phase 1.

## Environment

Same Ubuntu VM; /var/log/auth.log and /var/log/ufw.log as primary evidence sources.

## Commands Used

```
ls -la /var/log/
```

→ Lists all available system log sources on the host.

```
sudo tail -50 /var/log/auth.log
```

→ Reviews the most recent authentication log entries.

```
sudo grep -ai "invalid user\|failed password" /var/log/auth.log
```

→ Searches auth.log for failed or invalid login attempts (-a forces text mode).

```
sudo tail -30 /var/log/ufw.log
```

→ Reviews recent firewall allow/block activity.

```
sudo grep -i "block" /var/log/ufw.log | tail -20
```

→ Filters the firewall log specifically for blocked connection attempts.

```
arp -a
```

→ Lists devices currently visible on the local network via the ARP cache.

## Findings

- auth.log showed a clean, fully-attributed timeline of the Phase 1 remediation work — every sudo command, the SSH restart sequence, and the first successful publickey login, with no unexplained entries.
- ufw.log surfaced a one-time blocked FTP connection (port 21) from an unfamiliar host (192.168.100.10) several days prior, alongside a high volume of benign WS-Discovery multicast background noise from Docker/VirtualBox networking.
- ARP cache no longer held an entry for 192.168.100.10 by the time of analysis — the 5-day gap meant the device could not be conclusively identified.

## Remediation / Outcome

- Documented the FTP finding honestly as inconclusive rather than guessing, with a recommendation to correlate firewall logs with DHCP/ARP data in near real time during live investigations.

# Phase 3 — SIEM / SOC Investigation (Wazuh)

## Objective

Move from raw log files to Wazuh's SIEM dashboard, investigate alerts through correlation across data sources, and validate the SIEM's own coverage.

## Environment

Wazuh 4.9.0 in Docker (wazuh-manager, wazuh-indexer, wazuh-dashboard containers), Suricata IDS, dashboard accessed at <https://192.168.100.20>.

## Commands Used

```
sudo docker ps
```

→ Confirms the Wazuh manager, indexer, and dashboard containers are running.

```
sudo docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/agent_control -l
```

→ Lists registered Wazuh agents and their live connection status.

```
Dashboard query: data.alert.signature_id:1000001
```

→ Searches Wazuh/Suricata alerts by a specific signature ID.

```
Dashboard query: data.dest_port:6200
```

→ Checks for any traffic on the known vsftpd 2.3.4 backdoor shell port.

```
sudo tail -30 /var/ossec/logs/ossec.log
```

→ Reviews Wazuh's own internal operational log for connection/processing issues.

```
sudo grep -i "auth.log" /var/ossec/etc/ossec.conf
```

→ Checks whether auth.log is configured as a monitored log source (returned empty — root cause found).

```
sudo nano /var/ossec/etc/ossec.conf (add <localfile> block for /var/log/auth.log)
```

→ Adds the missing log source so SSH/auth activity is actually ingested by Wazuh.

```
sudo systemctl restart wazuh-agent
```

→ Restarts the agent to apply the new configuration and force reconnection.

```
Dashboard query: agent.name:jayy and location:"/var/log/auth.log"
```

→ Confirms real auth.log events are now flowing into Wazuh after the fix.

## Findings

- A Suricata alert ("LOCAL FTP vsftpd 2.3.4 backdoor trigger detected") matched the unexplained FTP traffic from Phase 2 — traced to an outbound connection from this VM to 192.168.100.10:21, consistent with a Metasploitable-style intentionally vulnerable target.
- Follow-up search for port 6200 (the backdoor's actual shell port) returned zero hits, confirming detection of the trigger pattern only — no evidence of a shell being opened.
- Critical gap discovered: `/var/log/auth.log` was never configured as a Wazuh-monitored source, meaning all SSH login and sudo activity from Phases 1–2 had been completely invisible to the SIEM despite the agent showing "Active."
- Recurring Wazuh agent disconnect/reconnect cycles observed in the manager's operational log, pointing to intermittent connectivity in the lab environment.

## **Remediation / Outcome**

- Added the missing `auth.log` block to `ossec.conf` and restarted the agent.
- Verified the fix with a live test login (invalid user "finaltest") — correctly detected by Wazuh rule 5710 within roughly 30–60 seconds, closing the coverage gap.

# Phase 4 — Incident Response

## Objective

Simulate a real attack scenario and run it through the full incident response lifecycle: detect, investigate, contain, recover, and report.

## Environment

Same Ubuntu VM and Wazuh stack as Phases 1–3, SSH already hardened to key-only auth.

## Commands Used

```
for i in 1 2 3 4 5; do ssh wronguser$i@192.168.100.20; done
```

→ Simulates a rapid brute-force SSH attempt using five invalid usernames.

```
Dashboard query: agent.name:jayy and rule.level >= 5
```

→ Searches Wazuh for medium/high-severity alerts related to the simulated attack.

```
Dashboard query: agent.name:jayy and full_log:wronguser
```

→ Searches raw ingested logs for any trace of the brute-force attempt.

```
sudo grep -i wronguser /var/log/auth.log
```

→ Confirms the attack occurred at the OS log level, independent of the SIEM.

```
sudo grep -ai "queue\|drop\|discard\|overflow" /var/ossec/logs/ossec.log
```

→ Checks Wazuh's internal logs for dropped or overflowed event queues.

```
sudo grep -A3 "client_buffer" /var/ossec/etc/ossec.conf
```

→ Checks the configured Wazuh agent buffer/queue size (containment check).

```
sudo grep "AllowUsers" /etc/ssh/sshd_config
```

→ Verifies SSH access remains restricted to the intended user (containment check).

```
sudo systemctl status ssh wazuh-agent --no-pager
```

→ Confirms both services are healthy following the incident (recovery check).

## Findings

- All 5 simulated brute-force attempts were correctly and immediately rejected by the Phase 1 SSH hardening — zero successful logins, confirmed in raw auth.log.
- Despite this, the attempt generated no Wazuh alert at any severity level, even though auth.log was actively being monitored at the time.

- Investigation traced the gap to repeated agent disconnect/reconnect cycles around the incident window, plus a historical (12-day-old) "message queue is full, log lines may be lost" warning — root cause documented as plausible but not conclusively proven.

## **Remediation / Outcome**

- Containment: confirmed queue capacity (5000, adequate) and AllowUsers restriction still active.
- Recovery: confirmed SSH and Wazuh agent both healthy and operating normally post-incident.
- Reported the detection gap as an open risk requiring further monitoring, rather than overstating confidence in an inconclusive root cause.

# Phase 5 — Local Security Audit (Windows 11)

## Objective

Audit local user accounts, privileges, password/lockout policy, and security event logs on a real Windows 11 machine. (Adapted from Active Directory Security, since Windows 11 cannot host AD Domain Services — that role requires Windows Server.)

## Environment

Windows 11 host machine (hostname DESKTOP-3OKNFSD), PowerShell (Administrator).

## Commands Used

```
Get-LocalUser | Select-Object Name, Enabled, LastLogon, PasswordRequired, PasswordExpires
```

→ Lists all local accounts with status, last logon, and password requirements.

```
Get-LocalGroupMember -Group "Administrators"
```

→ Lists every account with local administrator privileges.

```
Get-LocalUser -Name "Analyst","User1" | Select-Object Name, PasswordRequired, PasswordLastSet
```

→ Checks specific accounts for password requirements and last-set date.

```
Disable-LocalUser -Name "User1"
```

→ Disables an account found to require no password at all.

```
net accounts
```

→ Displays system-wide password age, length, and account lockout policy.

```
net accounts /uniquepw:5
```

→ Increases enforced password history from 2 to 5 unique passwords.

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} -MaxEvents 20 | Format-List TimeCreated, Message
```

→ Retrieves full detail on failed logon events (Event ID 4625) from the Security log.

## Findings

- Two accounts (Analyst, User1) confirmed by the system owner as intentional test accounts — not in the Administrators group.
- User1 was enabled with PasswordRequired = False and no PasswordLastSet date — a genuine, unremediated no-password local account.

- Password history policy was weak (2), allowing indefinite alternation between just two passwords. Minimum password length was 0, an accepted risk given primary reliance on Windows Hello facial recognition for daily login.
- Only two failed logon events (4625) in the Security log, both self-originated by system processes/the local account itself — no evidence of external or malicious login attempts.

## **Remediation / Outcome**

- Disabled User1 until genuinely needed.
- Increased password history enforcement from 2 to 5.
- Documented the minimum-password-length-0 setting as a deliberate, owner-accepted risk rather than an oversight, since Windows Hello is the primary authentication path.

# Phase 6 — Threat Hunting

## Objective

Proactively search for signs of compromise — unusual processes, network listeners, and recent file activity — without waiting for an alert to trigger.

## Environment

Same Ubuntu VM and Wazuh/Docker stack.

## Commands Used

```
ps aux --sort=-%cpu | head -20
```

→ Lists the top CPU-consuming processes, checking for unrecognized names or unusual run locations.

```
ps -eo pid,ppid,user,lstart,cmd --sort=-%cpu | head -20
```

→ Same view with process start times and parent PIDs for deeper context.

```
id dnsmasq / getent passwd dnsmasq
```

→ Investigates an unexpected process owner by checking the account's UID, GID, and login shell.

```
sudo ss -tulpn | grep -v 127.0.0.1
```

→ Lists all listening network ports, excluding localhost-only services, to check for unexplained exposure.

```
sudo find / -mmin -60 -type f -not -path "/proc/*" -not -path "/sys/*" -not -path  
"/var/lib/docker/*" -not -path "/var/log/*"
```

→ Searches for any file modified in the last 60 minutes outside expected noisy system paths.

## Findings

- Anomaly identified: several core Wazuh processes (analysisd, wazuh-db, syscheckd, remoted) appeared in `ps` output as owned by the "dnsmasq" user rather than root or a Wazuh-specific account.
- Investigation via id/getent confirmed dnsmasq's shell is /usr/sbin/nologin (no interactive login possible) and its UID (999) is a standard low system-service UID — consistent with a benign Docker UID-mapping display artifact, not a compromise.
- Every listening port mapped cleanly to a known service: SSH, the Wazuh stack (via docker-proxy), DNS (systemd-resolve), and NTP (chronyd) — no unexplained listeners.
- Recently modified files were limited to expected desktop-session and systemd journal housekeeping — nothing in an unusual location, nothing executable dropped unexpectedly.

## Remediation / Outcome

- No remediation required — the hunt concluded clean across all three angles (processes, network, filesystem), documented as a legitimate, evidence-based negative result rather than an assumed-safe skip.

# Phase 7 — Advanced Wireshark Investigation

## Objective

Capture and manually analyze raw packet traffic to directly observe protocol-level security weaknesses — specifically, credential exposure over an unencrypted protocol.

## Environment

Ubuntu VM, Wireshark 4.6.4 / tshark, a deliberately-enabled Telnet server for the demonstration.

## Commands Used

```
sudo apt install tshark -y
```

→ Installs the command-line Wireshark packet capture tool.

```
sudo apt install telnetd inetutils-inetd -y
```

→ Installs a Telnet server for the insecure-protocol demonstration.

```
sudo systemctl enable --now telnet.socket
```

→ Enables and starts the Telnet service via systemd socket activation.

```
sudo ss -tulpn | grep :23
```

→ Confirms the Telnet service is actively listening on port 23.

```
sudo tshark -i lo -w /tmp/phase7_lo.pcap
```

→ Captures live packet traffic on the loopback interface (used since client and server were the same machine).

```
telnet 192.168.100.20
```

→ Connects to the local Telnet service, generating a real plaintext login session.

```
wireshark /tmp/phase7_lo.pcap → filter: telnet → Follow → TCP Stream
```

→ Opens the capture and reconstructs the full Telnet session to inspect the plaintext credential exchange.

```
passwd
```

→ Changes the account password after it was intentionally exposed during the demonstration.

```
sudo systemctl disable --now telnet.socket && sudo systemctl stop inetutils-inetd && sudo systemctl disable inetutils-inetd
```

→ Fully disables the insecure Telnet service once the demonstration is complete.

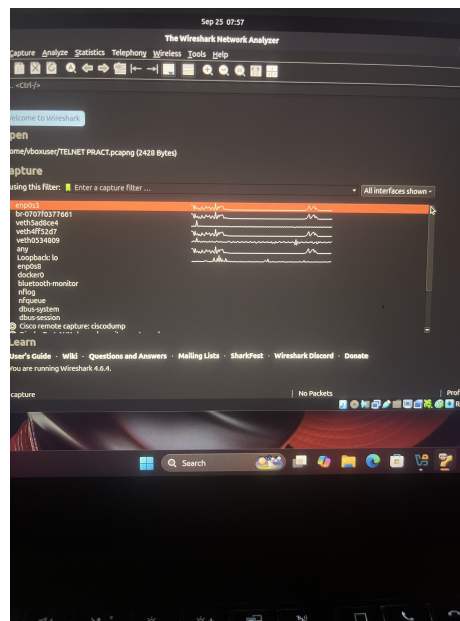
## Findings

- Successfully captured a complete Telnet login session, including full option-negotiation handshake.

- Follow TCP Stream reconstruction revealed the username and password in fully readable plaintext, character-by-character (Telnet echoes each keystroke as a separate packet), followed by the login banner and subsequent commands.
- Direct, first-hand confirmation of why Telnet must never be used for real administrative access, and concrete visual evidence of the value of the Phase 1 SSH key-only hardening by contrast.

## Remediation / Outcome

- Changed the exposed account password immediately after the demonstration.
- Disabled and removed the Telnet service entirely once the capture and analysis were complete, leaving no unnecessary insecure service running.



Wireshark 4.6.4 open on the Ubuntu VM, with the captured "TELNET PRACT.pcapng" session and available capture interfaces listed (enp0s8, enp0s3, docker0, loopback, and Docker veth pairs).

# Phase 8 — Final Capstone Summary

Phase 8 is not a new technical exercise — it is where the seven prior phases are tied into one coherent, connected investigation rather than treated as isolated labs. The full narrative appears as the Executive Summary at the front of this document.

## The Story, In Order

- **Recon & Vulnerability Assessment** (Phase 1) — scanned the environment, found a real CVE, patched what could be patched, hardened what couldn't.
- **Log Analysis** (Phase 2) — read raw evidence of that remediation, caught a real unexplained network anomaly.
- **SIEM Investigation** (Phase 3) — took that anomaly into Wazuh, traced it to a Metasploitable-style backdoor signature, and discovered/fixed a real SIEM blind spot.
- **Incident Response** (Phase 4) — simulated an attack, confirmed defenses held, documented an inconclusive-but-honest detection gap.
- **Local Security Audit** (Phase 5) — found and remediated a real no-password account on an actual Windows machine.
- **Threat Hunting** (Phase 6) — proactively searched for hidden compromise, found and correctly cleared one anomaly, documented a clean result.
- **Packet Analysis** (Phase 7) — proved why the Phase 1 SSH hardening mattered by showing Telnet's plaintext exposure directly.

## Closing Note

The consistent thread across all eight phases isn't that everything went perfectly — it's that every finding was investigated methodically, verified with evidence rather than assumption, and either remediated or honestly documented as unresolved. That discipline, more than any single tool used, is the core takeaway of this portfolio.